Hoist Grid Guide

Sample document for the Hoist PdfViewer demo

Content from the hoist-react `cmp/grid` README - github.com/xh/hoist-react

Grid Package

Section	Description
Overview	Introduction to Hoist's tabular and hierarchical data grid
Architecture	Class hierarchy and key classes
Configuration Pattern	Spec-to-object transformation for declarative config
Common Usage Patterns	Sorting, grouping, tree mode, selection, filtering, and more
Column Properties Reference	Key categories of ColumnSpec properties
Extension Points	ag-Grid passthrough, custom comparators, and appData
App-Level Defaults	Static defaults for GridModel and GridFilterModel
Pitfalls	Duplicate column IDs and other common issues
Related Packages	Links to data, ag-grid, desktop/mobile grid, and svc

Overview

Grid is Hoist React's primary component for displaying tabular and hierarchical data. Built on ag-Grid, it provides a consistent API with extensive customization for sorting, grouping, filtering, selection, inline editing, and export.

Architecture

Class Hierarchy

```
GridModel (orchestrator)
├── store: Store                  # Data source (from /data/)
├── selModel: StoreSelectionModel # Selection state
├── columns: Column[]             # Column definitions
├── filterModel: GridFilterModel  # Column-level filtering (desktop only)
├── colChooserModel: HoistModel   # Column visibility UI (modal or docked)
└── agGridModel: AgGridModel      # ag-Grid wrapper (from /cmp/ag-grid/)
```

Key Classes

- **GridModel** (`GridModel.ts`) - Central orchestrator managing sorting, grouping, selection, column state, and expansion. Created with `GridConfig` and provides the primary application API.

- **Column** (`columns/Column.ts`) - Individual column definition with ~50 configurable properties including rendering, sorting, filtering, editing, and export behavior. Applications provide `ColumnSpec` configs; Hoist creates `Column` instances internally.
- **ColumnGroup** (`columns/ColumnGroup.ts`) - Hierarchical column organization for multi-level headers. Configured via `ColumnGroupSpec`.
- **GridSorter** (`GridSorter.ts`) - Sort specification supporting ascending/descending and absolute value sorting. Includes a static `defaultComparator` for consistent null handling.
- **GridFilterModel** (`filter/GridFilterModel.ts`) - Manages column-level filters, integrating with Hoist's `Filter` system from `/data/`. Desktop only.

Configuration Pattern

Grid uses a **Spec** → **Object** transformation:

- `ColumnSpec` (plain config object) → `Column` (runtime instance)
- `ColumnGroupSpec` → `ColumnGroup`
- `GridConfig` → `GridModel`

This allows declarative configuration while providing rich runtime APIs.

Common Usage Patterns

Basic Grid Setup

```
import {grid, GridModel} from '@xh/hoist/cmp/grid';

const gridModel = new GridModel({
    store: {
        fields: ['name', 'value', 'date']
    },
    columns: [
        {field: 'name', flex: 1},
        {field: 'value', ...number},
        {field: 'date', ...date}
    ],
    sortBy: 'name'
});

// In component
grid({model: gridModel});
```

Pre-built Column Specs

Import partial column configs and spread into your column definitions:

```
import {number, boolCheck, date, dateTime, localDate, tags} from '@xh/hoist/cmp/grid';

columns: [
    {field: 'price', ...number},          // Right-aligned with number formatting
    {field: 'active', ...boolCheck},      // Green check for truthy values
    {field: 'created', ...date},          // Date formatting and Excel export
    {field: 'modified', ...dateTime},     // DateTime formatting
    {field: 'categories', ...tags}        // Tag pill rendering
]
```

Sorting

```
// Initial sort via config
new GridModel({
    sortBy: 'name',                          // Single column
    sortBy: ['name', {colId: 'value', sort: 'desc'}],  // Multi-column
    sortBy: {colId: 'value', sort: 'desc', abs: true}  // Absolute value sort
});

// Programmatic sort
gridModel.setSortBy([{colId: 'value', sort: 'desc'}]);

// Column-level sort order (cycles on header click)
{field: 'value', sortingOrder: ['desc', 'asc', null]}  // desc first, then asc, then
clear
{field: 'value', absSort: true}  // Enable absolute value sorting option
```

Grouping

```
// Full-width row grouping by column
new GridModel({
    groupBy: 'category',           // Single level
    groupBy: ['region', 'category'], // Multi-level
    expandLevel: 1                 // Expand first level by default
});

// Programmatic grouping
gridModel.setGroupBy(['region']);
gridModel.expandAll();
gridModel.collapseAll();
gridModel.expandToLevel(2);
```

Column Groups

```
new GridModel({
    columns: [
        {field: 'name', flex: 1},
        {
            headerName: 'Q1',
            collapsed: true,                   // Render collapsed until the user
expands
            children: [
                {field: 'q1Jan', groupShowMode: 'expanded'},
                {field: 'q1Feb', groupShowMode: 'expanded'},
                {field: 'q1Total', groupShowMode: 'collapsed'}
            ]
        }
    ]
});
```

`groupShowMode` is what makes a group expandable, and it takes a mix of values to do so: the group's children must resolve to at least one column shown while expanded *and* one shown while collapsed. Columns default to `'always'`, shown in either state. A group of always-shown columns is a static header, as is one where every child specifies the same value. This is evaluated over currently-visible children only, so hiding columns via the chooser can leave a group non-expandable.

Expand/collapse state is tracked on `GridModel.columnGroupState`, one entry per configured group. Read it with `isColumnGroupExpanded(groupId)`, drive it with `setColumnGroupExpanded()` or `setColumnGroupState()`, and persist it via `persistWith` (on by default, alongside column state - see `GridModelPersistOptions.persistColumnGroups`).

Collapsing affects display only: `columnState`, `isColumnVisible()`, and export are all unaffected.

Tree Mode

```
new GridModel({
    treeMode: true,
    store: {loadTreeData: true},  // Store expects hierarchical data with `children`
    columns: [
        {field: 'name', isTreeColumn: true, flex: 1},  // Exactly one column hosts tree
controls
        {field: 'value', ...number}
    ],
    treeStyle: 'highlights'  // 'highlights' | 'borders' | 'none'
});
```

Selection

```
new GridModel({
    selModel: 'single',    // Default for desktop
    selModel: 'multiple',  // Multi-select with shift/ctrl
    selModel: 'disabled'   // Default for mobile
});

// Programmatic selection
await gridModel.selectAsync(recordOrId);
await gridModel.selectFirstAsync();
await gridModel.preSelectFirstAsync();  // Only if nothing selected
gridModel.clearSelection();

// Read selection - all properties are observable
gridModel.selectedRecord;   // Single record or null
gridModel.selectedRecords;  // Array of records
gridModel.hasSelection;     // Boolean
```

Filtering

```
// Enable column header filters (desktop only)
new GridModel({
    filterModel: true,  // Default config
    filterModel: {
        bind: store,    // Filter target (defaults to grid's store)
        commitOnChange: true
    },
    columns: [
        {field: 'status', filterable: true},  // Enable filter on specific columns
        {field: 'name', filterable: true}
    ]
});
```

Inline Editing

```
new GridModel({
    columns: [
        {
            field: 'name',
            editable: true,  // Or function: ({record}) => record.data.canEdit
            editor: textEditor()  // From @xh/hoist/desktop/cmp/grid
        }
    ],
    clicksToEdit: 2  // Double-click to edit (default)
});

// Programmatic editing
await gridModel.beginEditAsync({record: rec, colId: 'name'});
await gridModel.endEditAsync();
```

Export

```
new GridModel({
    enableExport: true,
    exportOptions: {filename: 'my-data'}
});

// Server-side export (uses GridExportService)
await gridModel.exportAsync({type: 'excel'});

// Client-side export (ag-Grid native)
gridModel.localExport('my-data', 'csv');
```

Custom Renderers

Use `xxxRenderer` factory functions (e.g., `numberRenderer`) when passing a statically configured renderer directly to the `renderer` config - the factory returns a reusable function. When rendering dynamically based on record data or otherwise customizing per-cell, call the underlying formatter directly (e.g., `fmtNumber`) to avoid creating a new function on each render. See `/format/README.md` for the full formatter and renderer API.

```
import {numberRenderer, fmtNumber} from '@xh/hoist/format';
import {Icon} from '@xh/hoist/icon';

columns: [
    // Configured formatter - passes options to the underlying format function
    {
        field: 'price',
        renderer: numberRenderer({precision: 2, prefix: '$'})
    },

    // Renderer function returning different icons based on value
    {
        field: 'status',
        renderer: (status) => {
            switch (status) {
                case 'active': return Icon.checkCircle({className: 'xh-green'});
                case 'pending': return Icon.clock({className: 'xh-orange'});
                case 'inactive': return Icon.x({className: 'xh-red'});
                default: return null;
            }
        }
    },

    // Renderer accessing multiple record fields - requires rendererIsComplex flag
    {
        field: 'value',
        rendererIsComplex: true,
        renderer: (value, {record}) => {
            const {currency} = record.data;
            return fmtNumber(value, {precision: 2, label: currency});
        }
    }
]
```

Tooltips

Hoist styles the tooltip content it renders itself - a plain value or string, and the validation messages shown on an editable cell - with `xh-grid-tooltip-frame` (background, border, radius, padding, max-width) and `xh-grid-tooltip--prewrap` (honors `\n` line breaks while still wrapping at max-width).

A `tooltip` that returns an **element** is left unstyled, so that a custom tooltip can supply its own chrome. To take Hoist's frame instead, add `xh-grid-tooltip-frame` to your own root:

```
{
    field: 'volume',
    tooltip: volume =>
        vbox({
            className: 'xh-grid-tooltip-frame',
            items: [fmtNumberTooltip(volume), div('Unusually high volume')]
        })
}
```

`xh-grid-tooltip-frame` is a standalone utility class, usable on any element - a custom tooltip is not nested in anything that provides the frame for it. (`--prewrap` is an ordinary modifier that Hoist applies to its own tooltips; set `white-space` directly if a custom tooltip needs it.)

Cell Corner Flags

`cellFlag` marks a cell with a small triangle in its top-right corner, in the color of a standard Hoist `Intent` - a compact alternative to spending a column or restyling the cell. Called per record, returning an `Intent` or `null`.

```
columns: [
    {
        field: 'volume',
        cellFlag: volume => (volume >= 9_000_000_000 ? 'warning' : null)
    },
    {
        field: 'price',
        cellFlag: (value, {record}) => (record.data.isStale ? 'warning' : null)
    }
]
```

One flag renders per cell, and on an editable column a failing validation always wins. Keep the function cheap - it runs once per candidate `Intent` on each rendered cell and is deliberately uncached, so a flag still reflects state that changes without the record, such as an async validation result. Size follows the grid's `sizingMode` via the `--xh-grid-cell-flag-size` custom property. Flags are CSS pseudo-elements, so they add no width and do not appear in grid exports.

Context Menus

`GridModel.contextMenu` takes an array of entries, or a function returning one. Entries can be a `RecordAction` (or its config), a `'-'` separator, a `MenuHeading`, or a token - either one of Hoist's own (`copyCell`, `colChooser`, `export`, `filter`, `restoreDefaults`, and others) or one built into ag-Grid.

A `MenuHeading` is a non-interactive label for the items below it, written as `{heading: '...'}`. Use it to break a long menu into sections.

```
contextMenu: [
    {heading: 'This Row'},
    'copyCell',
    editAction,
    deleteAction,

    {heading: 'Grid'},
    'colChooser',
    'export',
    'restoreDefaults'
]
```

Headings render their own divider rule, so they need no `'-'` alongside. A heading with nothing below it is dropped automatically - at the end of a menu, or when its entire section has hidden itself. That matters because actions commonly hide based on the clicked row, via `hidden`, `recordsRequired`, or `displayFn`: right-clicking empty space in the grid above drops the `This Row` heading along with the actions under it.

A heading takes a `displayFn` of its own for dynamic text, receiving the same `ActionFnData` passed to the actions beside it:

```
{
    heading: 'This Row',
    displayFn: ({record}) => (record ? {heading: `Row: ${record.get('company')}`} :
{hidden: true})
}
```

Headings are also accepted by the desktop and mobile menus that take `MenuItemLike` entries - e.g. a dropdown attached to a button. Those menus supply no record context, so a `displayFn` there is called with no argument.

Column Properties Reference

Every column within a `GridModel` must resolve to a **unique ID**. The `colId` defaults to `field` when not explicitly set — so if two columns share the same `field` name (even across column groups), they must specify distinct `colId` values. Similarly, `ColumnGroup.groupId` defaults to `headerName` and must also

be unique across the grid. GridModel validates this at construction time and will throw if duplicates are detected.

Key categories of `ColumnSpec` properties:

Category	Properties
Identity	`field`, `colId` (unique), `displayName`, `description`
Display	`headerName`, `headerTooltip`, `width`, `flex`, `minWidth`, `maxWidth`, `hidden`, `align`, `groupShowMode`
Sorting	`sortable`, `sortingOrder`, `absSort`, `sortValue`, `sortToBottom`, `comparator`
Filtering	`filterable`
Editing	`editable`, `editor`, `editorIsPopup`
Export	`exportName`, `exportValue`, `excludeFromExport`, `excelFormat`, `excelWidth`
Chooser	`chooserName`, `chooserGroup`, `chooserDescription`*, `excludeFromChooser`, `hideable`
Rendering	`renderer`, `rendererIsComplex`, `tooltip`, `cellClass`, `cellClassRules`, `cellFlag`
Tree	`isTreeColumn`, `headerHasExpandCollapse`
Autosize	`autosizable`, `autosizeIncludeHeader`, `autosizeIncludeHeaderIcons`, `autosizeMinWidth`, `autosizeMaxWidth`

* `description` defaults from `Field.description` and serves as the default for both `headerTooltip` and `chooserDescription` when those are not explicitly set.

Extension Points

ag-Grid Passthrough

Both `Column` and `ColumnGroup` support an `agOptions` property for direct ag-Grid configuration:

```
{
    field: 'notes',
    agOptions: {
        wrapText: true,
        autoHeight: true
    }
}
```

`GridModel` does not expose a top-level `agOptions`, but the underlying `AgGridModel` is accessible for advanced use cases.

Custom Comparators

```
{
    field: 'priority',
    comparator: (valueA, valueB, sortDir, abs, {recordA, recordB, defaultComparator}) =>
{
        // Custom sort logic
        const priorityOrder = {high: 0, medium: 1, low: 2};
        return priorityOrder[valueA] - priorityOrder[valueB];
    }
}
```

appData

Both `GridModel` and `Column` support an `appData` property for storing custom application data:

```
new GridModel({
    appData: {source: 'user-list'},
    columns: [{field: 'name', appData: {searchable: true}}]
});
```

App-Level Defaults

GridModel and GridFilterModel expose a `static defaults` object for app-wide configuration overrides. Set these at app startup (e.g. in your `AppModel` constructor) to change framework defaults for all grids. Instance-level config always takes precedence.

`GridModel.defaults` covers a wide range of visual, behavioral, and structural properties — including `enableExport`, `showHover`, `rowBorders`, `stripeRows`, `cellBorders`, `headerMenuDisplay`, `colDefaults`, `exportOptions`, and more. See the `GridModelDefaults` interface for the full typed list with all available properties.

`GridFilterModel.defaults` provides `activeFilterIcon` to customize the icon displayed in column headers when a filter is active.

Example

```
import {GridModel} from '@xh/hoist/cmp/grid';
import {GridFilterModel} from '@xh/hoist/cmp/grid';
import {Icon} from '@xh/hoist/icon';

// In AppModel constructor – set app-wide grid preferences
GridModel.defaults.enableExport = true;
GridModel.defaults.showHover = true;
GridModel.defaults.rowBorders = true;
GridModel.defaults.headerMenuDisplay = 'hover';
GridModel.defaults.colDefaults = {sortable: false};
GridModel.defaults.contextMenu = [
    ...GridModel.defaults.contextMenu,
    '-',
    'myCustomAction'
];
GridFilterModel.defaults.activeFilterIcon = Icon.filter({prefix: 'fas', intent:
'warning'});
```

Pitfalls

Duplicate Column IDs

GridModel requires that every column and column group resolves to a unique ID. Column IDs default to `field` names, so assembling columns from multiple sources (e.g. concatenating arrays of field names or spreading shared column sets) can easily produce duplicates. GridModel will throw at construction time with a `Non-unique ids` error if this happens.

To fix, either remove the duplicate column or provide an explicit `colId` on one of the conflicting columns:

```
const detailColumns = [{field: 'name'}, {field: 'status'}];
const metricColumns = [{field: 'name'}, {field: 'value'}];

// BAD – 'name' appears in both arrays, producing duplicate colIds
columns: [...detailColumns, ...metricColumns]

// GOOD – deduplicate by field before passing to GridModel
columns: uniqBy([...detailColumns, ...metricColumns], 'field')
```

Related Packages

- `/data/` - Store, StoreRecord, Field, Filter infrastructure

- `/cmp/ag-grid/` - AgGridModel wrapper
- `/desktop/cmp/grid/` - Desktop-specific components (editors, column chooser dialogs)
- `/mobile/cmp/grid/` - Mobile-specific components
- `/svc/` - GridExportService, GridAutosizeService